Networks, signals, and parameters

ar084 · 14 August 2026 · Draft

Contents

Developer guide

Networks

`Network` is the mutable authoring object. Compilation turns it into immutable graph data. Every graph has a name and timestep.

```
net = snn.Network("example", dt=0.1 * snn.ms)
```

Names must be unique and contain no whitespace. They become stable identifiers in graphs, recordings, and checkpoints.

Signals and inputs

Signals carry an identifier, shape, physical unit, and signal type. Time-varying inputs use the canonical `(time, batch, channels)` axis order.

```
image = net.input(
    "image",
    shape=("time", "batch", 784),
    signal_type="spikes",
    unit="spike",
)
```

This declares what the graph accepts. It does not decide whether the spikes came from MNIST, SHD, a Poisson encoder, or a recorded event stream.

Populations

A population contains equally configured neurons and exposes named ports such as `spikes`, `voltage`, `excitatory`, and `inhibitory`.

```
excitatory = net.population(
    "E",
    size=80,
    neuron=snn.COBA_LIF(
        capacitance_nf=1.0,
        leak_us=0.05,
        resting_mv=-65.0,
        threshold_mv=-50.0,
        reset_mv=-65.0,
        refractory_steps=12,
    ),
)
```

The authoring API describes COBA-LIF, LIF, and non-spiking leaky-integrator populations. The graph executor currently supports COBA-LIF and leaky integrators. General current-based LIF execution is not implemented.

Parameters

Parameters have stable names, shapes, units, initializers, and optional constraints. Projections normally create their own dense weight parameter, but a named `ParameterRef` may be supplied explicitly.

Initializers explicitly distinguish lower-clamped normal, signed normal, uniform, constant, and zero distributions. Lower-clamped normal may request Bernoulli or exact-fan-in initial zeroing; those zeros remain ordinary parameters rather than a permanent connectivity mask. Projection weights record fan-in normalization, while direct readout parameters record direct storage. The execution result reports each parameter's initializer, constraint, unit, runtime shape, scaling rule, and realized count, zero fraction, mean, standard deviation, minimum, and maximum.

API reference

`Network`

```
Network(name: str, *, dt: Quantity = 0.1 * ms)
```

`name` becomes the graph identifier. `dt` is a time `Quantity`. Names claimed inside one network must be non-empty, unique, and contain no whitespace.

`Network.input`

```
net.input(name, *, shape, signal_type, unit="1") -> Signal
```

`shape` is a tuple of integers and symbolic axes. Time-varying dense inputs begin with `("time", "batch", ...)`. The returned signal identifier is `<name>.value`.

`Network.population`

```
net.population(name, *, size, neuron, spiking=True) -> Population
```

`size` must be positive. A spiking population exposes `.spikes` and `.voltage`; a non-spiking population exposes `.voltage`. Target ports are `.excitatory`, `.inhibitory`, and `.modulatory`.

`Network.parameter` and `Network.constant`

```
net.parameter(name, *, shape, initializer, unit="1", constraint=None) ->
ParameterRef
net.constant(name, value, *, unit="1") -> str
```

Parameter constructors are `LowerClampedNormal(mean, std, initial_zero_fraction=0, zeroing="bernoulli")`, its compatibility spelling `Normal(mean, std)`, `SignedNormal(mean, std)`, `Uniform(low, high)`, `Constant(value)`, and `Zeros()`. Exact fan-in zeroing uses `zeroing="exact_k"`. `NonNegative()` is the implemented constraint. Neuron constructors are `COBA_LIF(**values)`, `LIF(**values)`, and `LeakyIntegrator(**values)`. Unit helpers are `ms`, `mV`, `nS`, and `Hz`.

Core return types

`Signal` exposes `id`, `shape`, `unit`, `signal_type`, `owner`, and `port`. `Population` exposes `id`, `size`, `neuron`, `spiking`, `group`, and its signal or target-port properties. `ParameterRef` contains the stable parameter `id`.

Next: Components, projections, and delays